

## Experiment 1: Hadoop HDFS Basics

### Aim

To understand and perform basic file system operations in Hadoop Distributed File System (HDFS), such as creating directories, uploading files, listing contents, and retrieving or deleting files from HDFS.

### Software Requirements

- **Operating System:** Ubuntu/Linux (Preferred)
- **Java JDK:** Version 8 or higher
- **Apache Hadoop:** Version 2.x or 3.x (single-node or pseudo-distributed mode)
- **Terminal/Command Line Interface**
- **Optional:** SSH, SCP, and basic Unix/Linux utilities

### Learning Objectives

By the end of this experiment, students will be able to:

1. Understand the role of HDFS in the Hadoop ecosystem.
2. Learn basic HDFS shell commands for file operations.
3. Apply command-line tools to interact with HDFS.
4. Differentiate between local file system and HDFS usage.

### Theory

HDFS (Hadoop Distributed File System) is the primary storage system used by Hadoop applications. It provides reliable, scalable, and distributed storage designed to run on commodity hardware. HDFS is optimized for high-throughput access to large datasets and follows a **write-once, read-many** model.

### Architecture:

- **NameNode** (Master): Maintains metadata like the namespace, file locations, block information, etc.
- **DataNode** (Slave): Stores the actual data blocks on physical disks and serves them to clients.

### Key Characteristics:

- Fault-tolerant via data replication (default: 3 copies)
- Optimized for large file storage and sequential access
- Supports parallel processing by storing data close to computation

### Common Use-Cases:

- Storage layer for big data analytics
- Distributed file management
- Data ingestion from various sources

## Code / Procedure

Below are essential HDFS commands with **syntax and examples**:

### 1. Create a Directory in HDFS

- **Syntax:**

```
hadoop fs -mkdir <hdfs-directory-path>
```

- **Example:**

```
hadoop fs -mkdir /user/hadoop/input
```

Creates an input folder under the Hadoop user directory in HDFS.

### 2. Upload (Put) a File to HDFS

- **Syntax:**

```
hadoop fs -put <local-source-path> <hdfs-destination-path>
```

- **Example:**

```
hadoop fs -put sample.txt /user/hadoop/input
```

Uploads sample.txt from the local file system to /user/hadoop/input in HDFS.

### 3. List Files/Directories in HDFS

- **Syntax:**

```
hadoop fs -ls <hdfs-directory-path>
```

- **Example:**

```
hadoop fs -ls /user/hadoop/input
```

Lists contents of the input directory in HDFS.

### 4. Read a File from HDFS

- **Syntax:**

```
hadoop fs -cat <hdfs-file-path>
```

- **Example:**

```
hadoop fs -cat /user/hadoop/input/sample.txt
```

Displays the content of the file.

#### 5. **Download (Get) a File from HDFS to Local System**

- **Syntax:**

```
hadoop fs -get <hdfs-file-path> <local-destination-path>
```

- **Example:**

```
hadoop fs -get /user/hadoop/input/sample.txt /home/user/
```

Copies file from HDFS to the local directory /home/user/.

#### 6. **Delete a File from HDFS**

- **Syntax:**

```
hadoop fs -rm <hdfs-file-path>
```

- **Example:**

```
hadoop fs -rm /user/hadoop/input/sample.txt
```

Removes sample.txt from HDFS permanently.

### **Conclusion**

In this experiment, the fundamental file operations on Hadoop Distributed File System were performed successfully. Students were able to navigate HDFS using command-line utilities, enhancing their understanding of how distributed file systems work in a big data environment.

### **Learning Outcomes**

After completing this lab, students will be able to:

- Execute basic HDFS commands to manage files and directories.
- Understand the architecture and working principles of HDFS.
- Differentiate between local storage and distributed HDFS storage.
- Apply this knowledge in further experiments involving Hive, MapReduce, and Spark.

## **Experiment 2: Hive and Descriptive Analytics**

### **Aim**

To create a Hive database and table, load structured data, and perform basic descriptive statistical analysis using Hive Query Language (HiveQL).

### **Software Requirements**

- Apache Hadoop (pre-installed and configured)
- Apache Hive
- Java JDK 8+
- Ubuntu/Linux OS (preferred)
- Sample CSV dataset (e.g., patients.csv)
- Terminal or Hive CLI

### **Learning Objectives**

By the end of this experiment, students will be able to:

1. Understand the fundamentals of Hive and its role in the Hadoop ecosystem.
2. Create and query Hive tables using HiveQL.
3. Perform basic statistical analysis such as count, average, min, and max using SQL-like queries.
4. Import structured data from a CSV file into Hive tables.

### **Theory**

Apache Hive is a data warehouse system built on top of Hadoop that enables users to query and manage large datasets using a SQL-like language called **HiveQL**. Hive translates queries into MapReduce jobs under the hood, abstracting away the complexity of distributed computation.

### **Key Components of Hive:**

- **Metastore:** Stores metadata about databases, tables, partitions.
- **Driver:** Manages the lifecycle of a HiveQL statement.
- **Compiler:** Converts HiveQL into MapReduce jobs.
- **Execution Engine:** Executes the query plan.

### **Why Hive?**

- Allows SQL developers to write queries on large-scale Hadoop data.
- Supports ETL operations.
- Ideal for batch processing and analysis.

## Descriptive Analytics in Hive

Descriptive analytics summarizes raw data into meaningful statistics such as:

- COUNT() – Total number of records.
- AVG() – Average of numeric columns.
- MIN() / MAX() – Smallest and largest values in a column.

### Code / Procedure

#### Step 1: Start Hive CLI

hive

#### Step 2: Create a Hive Database

```
CREATE DATABASE healthcare;  
USE healthcare;
```

#### Step 3: Create a Table for Patients

```
CREATE TABLE patients (  
  id INT,  
  name STRING,  
  age INT,  
  weight FLOAT  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```

#### Step 4: Load Data into Hive Table

Make sure your patients.csv file is present in the local system.

```
LOAD DATA LOCAL INPATH '/home/user/patients.csv' INTO TABLE patients;
```

#### Step 5: Perform Descriptive Analytics

- **Count total records:**

```
SELECT COUNT(*) FROM patients;
```

- **Average age of patients:**

```
SELECT AVG(age) FROM patients;
```

- **Minimum and Maximum weight:**

SELECT MIN(weight), MAX(weight) FROM patients;

- **Display all records:**

SELECT \* FROM patients;

## Conclusion

Hive provides an easy-to-use interface for processing structured data stored in HDFS. In this experiment, a database and table were created, and descriptive statistical functions were applied to analyze data using HiveQL. This illustrates how traditional SQL concepts integrate with big data platforms.

## Learning Outcomes

After completing this lab, students will be able to:

- Set up and use Hive for querying large datasets.
- Create Hive tables and load data into them.
- Execute SQL-like commands in Hive to perform statistical analysis.
- Apply Hive as a scalable alternative to traditional RDBMS for analytics.

### **Experiment 3: Word Count using MapReduce**

#### **Aim**

To implement and execute a Hadoop MapReduce program that counts the frequency of each word in a given input text file.

#### **Software Requirements**

- Apache Hadoop (configured in pseudo-distributed mode)
- Java Development Kit (JDK) 8 or higher
- Eclipse/VSCode or terminal-based text editor (for writing Java code)
- Ubuntu/Linux OS (preferred)
- Sample text file (e.g., input.txt)

#### **Learning Objectives**

By the end of this experiment, students will be able to:

1. Understand the concept of the MapReduce programming model.
2. Implement Mapper and Reducer logic using Java.
3. Execute a basic MapReduce job on Hadoop.
4. Interpret job outputs and logs from the HDFS output directory

#### **Theory**

MapReduce is a programming model for processing large-scale data in parallel on a Hadoop cluster. It breaks the job into two key phases:

- **Map phase:** Processes input and produces intermediate key-value pairs.
- **Reduce phase:** Aggregates key-value pairs from the mapper and produces the final result.

#### **Word Count Problem**

A classic example used to demonstrate the MapReduce model is counting the number of occurrences of each word in a large document.

#### **Code / Procedure**

##### **Step 1: Create Input File and Upload to HDFS**

Create a file input.txt with sample content:

Big data analytics is powerful. Big data helps businesses.

Upload it to HDFS:

```
hadoop fs -mkdir /user/hadoop/wordcount  
hadoop fs -put input.txt /user/hadoop/wordcount/
```

### Step 2: Create Mapper Class (TokenizerMapper.java)

```
public class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {  
    private final static IntWritable one = new IntWritable(1);  
    private Text word = new Text();  
  
    public void map(Object key, Text value, Context context) throws IOException,  
        InterruptedException {  
        StringTokenizer itr = new StringTokenizer(value.toString());  
        while (itr.hasMoreTokens()) {  
            word.set(itr.nextToken().toLowerCase());  
            context.write(word, one);  
        }  
    }  
}
```

### Step 3: Create Reducer Class (IntSumReducer.java)

```
public class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {  
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws  
        IOException, InterruptedException {  
        int sum = 0;  
        for (IntWritable val : values) {  
            sum += val.get();  
        }  
        context.write(key, new IntWritable(sum));  
    }  
}
```

### Step 4: Driver Class (WordCount.java)

```
public class WordCount {  
    public static void main(String[] args) throws Exception {  
        Configuration conf = new Configuration();  
        Job job = Job.getInstance(conf, "word count");  
        job.setJarByClass(WordCount.class);  
        job.setMapperClass(TokenizerMapper.class);  
        job.setCombinerClass(IntSumReducer.class);  
        job.setReducerClass(IntSumReducer.class);  
        job.setOutputKeyClass(Text.class);  
        job.setOutputValueClass(IntWritable.class);  
        FileInputFormat.addInputPath(job, new Path(args[0]));  
        FileOutputFormat.setOutputPath(job, new Path(args[1]));  
        System.exit(job.waitForCompletion(true) ? 0 : 1);  
    }  
}
```



### Step 5: Compile and Run

# Compile the code

```
javac -classpath `hadoop classpath` -d wordcount_classes *.java
```

# Create a jar

```
jar -cvf wordcount.jar -C wordcount_classes/ .
```

# Run the job

```
hadoop jar wordcount.jar WordCount /user/hadoop/wordcount/input.txt  
/user/hadoop/wordcount_output
```

### Step 6: Check Output

```
hadoop fs -cat /user/hadoop/wordcount_output/part-r-00000
```

### Example Output:

```
analytics 1  
big 2  
businesses. 1  
data 2  
helps 1  
is 1  
powerful. 1
```

### Conclusion

In this experiment, a MapReduce application was successfully implemented to count the frequency of each word in a text file. This demonstrated the capability of the Hadoop framework to handle distributed processing of large datasets using the Map and Reduce paradigms.

### Learning Outcomes

After completing this lab, students will be able to:

- Understand and explain the MapReduce programming model.
- Implement custom Mapper and Reducer classes in Java.
- Execute a MapReduce job on Hadoop.
- Interpret results generated from distributed data processing.

## **Experiment 4: Matrix Multiplication using MapReduce**

### **Aim**

To implement a Hadoop MapReduce program that performs multiplication of two matrices in a distributed and parallel environment.

### **Software Requirements**

- Apache Hadoop (version 2.x or 3.x)
- Java JDK 8+
- Linux/Ubuntu OS
- Eclipse/VSCode or terminal-based text editor
- Sample input matrix files (Matrix A and Matrix B)

### **Learning Objectives**

By the end of this experiment, students will be able to:

1. Understand how matrix operations are handled using the MapReduce paradigm.
2. Represent and transform matrix data as key-value pairs.
3. Implement Mapper and Reducer logic to perform multiplication of matrices.
4. Execute and test the matrix multiplication job on a Hadoop system.

### **Theory**

Matrix multiplication is a fundamental linear algebra operation that involves computing the dot product of rows from one matrix with columns from another. When multiplying two matrices A (of size  $m \times n$ ) and B (of size  $n \times p$ ), the resulting matrix C has the size  $m \times p$  and each element  $C[i][j]$  is calculated as:

$$C[i][j] = \sum_{k=1}^n A[i][k] * B[k][j]$$

In a distributed environment like Hadoop, matrices are treated as key-value records, and MapReduce allows parallel computation of each element of the result matrix.

### **Code / Procedure**

#### **Sample Input:**

**Matrix A (3x2)** – file: matrixA.txt

0,0,1  
0,1,2  
1,0,3  
1,1,4  
2,0,5  
2,1,6

**Matrix B (2x3)** – file: matrixB.txt

0,0,7  
0,1,8  
0,2,9  
1,0,10  
1,1,11  
1,2,12

Each row is formatted as:

<row\_index>,<column\_index>,<value>

### Step 1: Upload Input Files to HDFS

```
hadoop fs -mkdir /user/hadoop/matrix  
hadoop fs -put matrixA.txt /user/hadoop/matrix/  
hadoop fs -put matrixB.txt /user/hadoop/matrix/
```

### Step 2: Mapper Class (MatrixMapper.java)

```
public class MatrixMapper extends Mapper<LongWritable, Text, Text, Text> {  
    public void map(LongWritable key, Text value, Context context) throws IOException,  
        InterruptedException {  
        String[] tokens = value.toString().split(",");  
        String matrixName = tokens.length == 3 ? (tokens[0].equals("A") ? "A" : "B") : null;  
  
        int i = Integer.parseInt(tokens[0]);  
        int j = Integer.parseInt(tokens[1]);  
        int val = Integer.parseInt(tokens[2]);  
  
        if (value.toString().contains("matrixA")) {  
            for (int k = 0; k < 3; k++) {  
                context.write(new Text(i + "," + k), new Text("A," + j + "," + val));  
            }  
        } else {  
            for (int k = 0; k < 3; k++) {  
                context.write(new Text(k + "," + j), new Text("B," + i + "," + val));  
            }  
        }  
    }  
}
```

### Step 3: Reducer Class (MatrixReducer.java)

```
public class MatrixReducer extends Reducer<Text, Text, Text, IntWritable> {  
    public void reduce(Text key, Iterable<Text> values, Context context) throws IOException,  
        InterruptedException {  
        Map<Integer, Integer> mapA = new HashMap<>();  
        Map<Integer, Integer> mapB = new HashMap<>();
```

```
for (Text val : values) {
    String[] parts = val.toString().split(",");
    if (parts[0].equals("A")) {
        mapA.put(Integer.parseInt(parts[1]), Integer.parseInt(parts[2]));
    } else {
        mapB.put(Integer.parseInt(parts[1]), Integer.parseInt(parts[2]));
    }
}

int result = 0;
for (int k : mapA.keySet()) {
    if (mapB.containsKey(k)) {
        result += mapA.get(k) * mapB.get(k);
    }
}

context.write(key, new IntWritable(result));
}
}
```

#### Step 4: Driver Class (MatrixMultiply.java)

```
public class MatrixMultiply {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Matrix Multiplication");
        job.setJarByClass(MatrixMultiply.class);
        job.setMapperClass(MatrixMapper.class);
        job.setReducerClass(MatrixReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

#### Step 5: Compile, Package and Run the Job

```
javac -classpath `hadoop classpath` -d matrix_classes *.java
jar -cvf matrixmultiply.jar -C matrix_classes/ .
hadoop jar matrixmultiply.jar MatrixMultiply /user/hadoop/matrix
/user/hadoop/output_matrix
```

#### Step 6: Check Output

```
hadoop fs -cat /user/hadoop/output_matrix/part-r-00000
```

Expected Output (for each element of Matrix C):

0,0 27  
0,1 30  
0,2 33  
1,0 61  
1,1 68  
1,2 75  
2,0 95  
2,1 106  
2,2 117

## Conclusion

In this experiment, students implemented a parallelized matrix multiplication algorithm using Hadoop's MapReduce model. By decomposing the problem into key-value computations, the matrix product was successfully computed in a distributed fashion.

## Learning Outcomes

After completing this lab, students will be able to:

- Translate matrix operations into the MapReduce paradigm.
- Understand how Hadoop handles complex computations like multiplication.
- Implement and test distributed processing algorithms.
- Apply MapReduce to solve other linear algebra and data transformation problems.

## Experiment 5: DGIM Algorithm Implementation

### Aim

To implement the DGIM (Datar-Gionis-Indyk-Motwani) algorithm for approximating the number of 1s in the last  $N$  bits of a binary stream using limited memory.

### Software Requirements

- Python 3.x or Java (Python preferred for simplicity)
- Text editor (VSCode, Jupyter Notebook, etc.)
- Command line / terminal
- Optional: Matplotlib (for visualization)

### Learning Objectives

By the end of this experiment, students will be able to:

1. Understand the challenges of real-time stream processing.
2. Implement the DGIM algorithm to count the approximate number of 1s in the last  $N$  bits of a binary stream.
3. Analyze how space complexity can be reduced using approximation techniques.
4. Observe the trade-off between accuracy and memory usage.

### Theory

In big data environments, it's often not feasible to store entire data streams in memory. The **DGIM algorithm** provides a space-efficient way to estimate the count of recent events—in this case, the number of 1s in the last  $N$  bits of a stream.

### How DGIM Works:

- Uses a **bucket-based** approach.
- Buckets are of sizes in powers of 2 (1, 2, 4, 8...).
- Only keeps a **logarithmic number of buckets**.
- Combines older buckets to preserve space.

Each bucket represents a group of 1s. It stores:

- The **timestamp** (position in stream),
- The **size** of the bucket (number of 1s it represents).

When a new 1 arrives:

- A new bucket of size 1 is created.
- If more than two buckets of the same size exist, the oldest two are merged.

### Estimation:

To estimate the number of 1s in the last  $N$  bits, sum the sizes of the buckets that fall within the window. The oldest bucket's size is **halved** to maintain an upper bound.

## Code / Procedure (Python)

```
import math
```

```
class DGIM:
```

```
    def __init__(self, window_size):  
        self.N = window_size  
        self.buckets = [] # Each element: (timestamp, size)
```

```
    def _merge_buckets(self):  
        i = 0  
        while i < len(self.buckets) - 2:  
            if (self.buckets[i][1] == self.buckets[i+1][1] == self.buckets[i+2][1]):  
                # Merge the two oldest  
                self.buckets[i+1] = (self.buckets[i+1][0], self.buckets[i+1][1] * 2)  
                del self.buckets[i]  
            else:  
                i += 1
```

```
    def add_bit(self, bit, current_time):  
        if bit == 1:  
            self.buckets.insert(0, (current_time, 1))  
            self._merge_buckets()  
        # Remove buckets out of the window  
        self.buckets = [b for b in self.buckets if current_time - b[0] < self.N]
```

```
    def estimate(self):  
        total = 0  
        for i, (timestamp, size) in enumerate(self.buckets):  
            if i == len(self.buckets) - 1:  
                total += size / 2.0 # Halve the oldest  
            else:  
                total += size  
        return int(total)
```

```
# -----
```

```
# Simulate a stream of bits
```

```
stream = [1, 0, 1, 1, 0, 1, 0, 1, 1, 1]
```

```
dgim = DGIM(window_size=5)
```

```
for t, bit in enumerate(stream):
```

```
    dgim.add_bit(bit, t)
```

```
    print(f"Time {t}, bit={bit} => Estimated 1s in last 5 bits: {dgim.estimate()}")
```

## Conclusion

The DGIM algorithm was successfully implemented to process a binary stream in real time and approximate the number of 1s in the latest window of data. It demonstrated how space efficiency can be achieved without sacrificing too much accuracy in stream analytics.

## Learning Outcomes

After completing this lab, students will be able to:

- Implement real-time algorithms for streaming data.
- Approximate counts using logarithmic memory.
- Understand the importance of probabilistic algorithms in big data.
- Analyze trade-offs between precision and performance.



## Experiment 6: Bloom Filter Implementation

### Aim

To implement a Bloom Filter for efficient membership testing and understand its use in big data systems where approximate set membership queries are acceptable.

### Software Requirements

- Python 3.x or Java (Python recommended for simplicity)
- Text Editor or IDE (VSCode, Jupyter, etc.)
- Terminal/Command Line
- hashlib module (built-in in Python)

### Learning Objectives

By the end of this experiment, students will be able to:

1. Understand the concept of probabilistic data structures.
2. Implement a Bloom Filter with multiple hash functions.
3. Demonstrate the time-space trade-off in large-scale data systems.
4. Analyze false positives and understand their implications in real-world applications.

### Theory

A **Bloom Filter** is a space-efficient, probabilistic data structure used to test whether an element is a member of a set. It may return **false positives** (element may be reported present when it is not), but **never false negatives** (if reported absent, it is definitely absent).

### Working Principle:

- Uses a **bit array** of size  $m$ , initialized to 0.
- Uses  $k$  different **independent hash functions**.
- To add an element: Each hash function is applied, and the resulting index positions in the bit array are set to 1.
- To check membership: Apply the hash functions again. If all corresponding bits are 1, it *might* be in the set; if any bit is 0, it's definitely not.

### Trade-offs:

- Reduces space complexity drastically.
- Fast insertion and query operations ( $O(k)$ ).
- Comes with the risk of **false positives**, controlled by  $m$  and  $k$ .

### Code / Procedure (Python)

```
import hashlib
```

```
class BloomFilter:
```

```
    def __init__(self, size=100, hash_count=3):
```

```
        self.size = size
```

```
        self.bit_array = [0] * size
```

```
        self.hash_count = hash_count
```

```
    def _hashes(self, item):
```

```
        hashes = []
```

```
        for i in range(self.hash_count):
```

```
            digest = hashlib.sha256((str(i) + item).encode('utf-8')).hexdigest()
```

```
            index = int(digest, 16) % self.size
```

```
            hashes.append(index)
```

```
        return hashes
```

```
    def add(self, item):
```

```
        for index in self._hashes(item):
```

```
            self.bit_array[index] = 1
```

```
    def check(self, item):
```

```
        return all(self.bit_array[index] == 1 for index in self._hashes(item))
```

```
# -----
```

```
# Example Usage:
```

```
bf = BloomFilter(size=50, hash_count=3)
```

```
# Adding elements
```

```
elements = ['apple', 'banana', 'orange']
```

```
for elem in elements:
```

```
    bf.add(elem)
```

```
# Checking membership
```

```
test_items = ['apple', 'banana', 'grape', 'mango']
```

```
for item in test_items:
```

```
    result = bf.check(item)
```

```
    print(f"{item}: {'Possibly Present' if result else 'Definitely Not Present'}")
```

## Conclusion

The Bloom Filter was successfully implemented and tested for set membership operations. It showed how we can significantly reduce memory usage while tolerating a small probability of error (false positives). This makes it a suitable choice for large-scale systems such as spam filters, web caching, and network intrusion detection.

## Learning Outcomes

After completing this lab, students will be able to:

- Design and implement probabilistic data structures.
- Evaluate trade-offs between accuracy and performance.
- Apply Bloom Filters to practical big data applications.
- Analyze the effects of false positives in large-scale systems.

## Experiment 7: Flajolet-Martin Algorithm

### Aim

To implement the Flajolet-Martin (FM) algorithm to estimate the number of distinct elements (cardinality) in a large data stream using probabilistic techniques.

### Software Requirements

- Python 3.x or Java (Python preferred)
- Terminal/Command Line
- Text Editor or IDE (e.g., Jupyter, VSCode)
- Built-in hashlib module in Python

### Learning Objectives

By the end of this experiment, students will be able to:

1. Understand the concept of cardinality estimation in data streams.
2. Implement the Flajolet-Martin algorithm for distinct count approximation.
3. Analyze the trade-off between accuracy and memory usage in big data.
4. Apply hash-based techniques for real-time analytics.

### Theory

The **Flajolet-Martin algorithm** is a streaming algorithm that estimates the **number of distinct elements** in a data stream using a small, fixed amount of memory.

### Concept Overview:

- Hash each incoming element to a binary string.
- Count the **maximum number of trailing zeros** in any hashed value seen so far.
- If the maximum number of trailing zeros is  $R$ , the estimated number of distinct elements is roughly  $2^{R+1}$ .

### Why It Works:

- Uniformly hashed values produce trailing zeros with a probability that decreases exponentially.
- More unique items  $\rightarrow$  greater chance of encountering a hash with many trailing zeros.

### Improvement:

To improve accuracy, **multiple hash functions** or **multiple estimators** are used and averaged or median values are taken.

## Code / Procedure (Python)

```
import hashlib
import random

def hash_value(x):
    return hashlib.sha1(str(x).encode()).hexdigest()

def get_trailing_zeros(binary_str):
    return len(binary_str) - len(binary_str.rstrip('0'))

def flajolet_martin(stream):
    max_zeros = 0
    for item in stream:
        hashed = int(hash_value(item), 16)
        binary = bin(hashed)[2:]
        trailing_zeros = get_trailing_zeros(binary)
        max_zeros = max(max_zeros, trailing_zeros)
    return 2 ** max_zeros

# -----
# Simulated Stream Input
stream = ['apple', 'banana', 'apple', 'grape', 'banana', 'mango', 'kiwi', 'apple']

# Estimate distinct count
estimate = flajolet_martin(stream)

print("Estimated number of distinct elements:", estimate)
print("Actual number of distinct elements:", len(set(stream)))
```

## Conclusion

The Flajolet-Martin algorithm was successfully implemented to estimate the number of distinct elements in a data stream. The result demonstrates the usefulness of probabilistic algorithms in scenarios where full storage of data is impractical or impossible.

## Learning Outcomes

After completing this lab, students will be able to:

- Apply streaming algorithms for approximate analytics.
- Implement and validate the Flajolet-Martin algorithm.
- Understand the role of hash functions in big data.
- Use space-efficient methods to derive insights from massive data streams.

## **Experiment 8: Dashboard using PowerBI/Tableau**

### **Aim:**

To create an interactive healthcare dashboard using visualization tools like PowerBI or Tableau for better understanding and analysis of healthcare datasets.

### **Software Requirements:**

- PowerBI Desktop (free) or Tableau Public/Professional
- Healthcare dataset (CSV, Excel, or database)

### **Theory:**

Business Intelligence (BI) tools transform raw data into interactive dashboards, enabling visual exploration of complex datasets. In healthcare, dashboards facilitate monitoring patient statistics, resource allocation, and treatment outcomes through charts, KPIs, and filters.

### **Procedure:**

1. **Import Dataset:**
  - Open PowerBI or Tableau.
  - Connect to the healthcare dataset (e.g., CSV file containing patient data).
  - Clean or transform data if needed (Power Query in PowerBI or Data Source tab in Tableau).
2. **Create Visualizations:**
  - Design different charts:
    - Bar Chart for patient counts per disease
    - Line Chart for trend analysis of admissions over time
    - Pie Chart for distribution of patients by gender or age group
  - Create KPIs such as average length of hospital stay or mortality rate.
3. **Add Interactivity:**
  - Use slicers (PowerBI) or filters (Tableau) for date range, disease, or demographics.
  - Enable drill-down on charts to explore detailed information.
4. **Build Dashboard:**
  - Combine all charts and KPIs on one dashboard page.
  - Arrange visuals for clear insight flow.
5. **Publish & Share:**
  - Publish on PowerBI Service or Tableau Public for sharing.

**Code/Syntax/Examples:**

*PowerBI DAX examples:*

-- Total patients admitted

TotalPatients = COUNTROWS('PatientData')

-- Average length of stay

AvgLengthOfStay = AVERAGE('PatientData'[LengthOfStay])

-- Mortality rate percentage

```
MortalityRate = DIVIDE(
    COUNTROWS(FILTER('PatientData', 'PatientData'[Outcome] = "Deceased")),
    COUNTROWS('PatientData'),
    0
) * 100
```

*Tableau Calculated Field example:*

// Percent of patients with diabetes

SUM(IF [Diagnosis] = "Diabetes" THEN 1 ELSE 0 END) / COUNT([PatientID])

**Conclusion:**

An interactive healthcare dashboard was created successfully using PowerBI/Tableau, allowing visualization of patient demographics, disease trends, and key healthcare metrics in a user-friendly way.

**Learning Outcomes:**

- Students learn how to import and visualize large datasets.
- Students gain experience creating interactive dashboards for healthcare analytics.
- Students develop skills in interpreting data visually for decision-making.

## **Experiment 9: Case Study on Real-Life Big Data Application**

### **Aim:**

To research and analyze a real-life big data application in any domain to understand its architecture, tools, challenges, and business impact.

### **Software Requirements:**

- None (Research-based)
- Access to online research articles, case studies, or reports.

### **Theory:**

Big data solutions leverage technologies like Hadoop, Spark, NoSQL databases, and cloud platforms to handle massive, complex datasets. Each industry (healthcare, finance, retail) uses big data uniquely, facing challenges like volume, velocity, and variety while aiming to extract actionable insights.

### **Procedure:**

1. **Choose a Domain:**  
Select an industry such as healthcare, banking, e-commerce, or IoT.
2. **Identify a Big Data Application:**  
Research a specific big data use case (e.g., fraud detection in banking, predictive healthcare analytics).
3. **Study Architecture and Tools:**
  - Identify tools used (e.g., Hadoop HDFS for storage, Spark for processing).
  - Understand data ingestion pipelines and analytics approaches.
4. **Analyze Challenges and Impact:**
  - Explore challenges like data privacy, processing speed, and scalability.
  - Assess business value and improvements achieved.
5. **Document and Present Findings:**  
Prepare a detailed report or presentation summarizing the application, technologies, challenges, and outcomes.

### **Code/Syntax/Examples:**

*Sample Kafka Producer Code (Java) to simulate data ingestion:*

```
Properties props = new Properties();  
props.put("bootstrap.servers", "localhost:9092");  
props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");  
props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");
```

```
Producer<String, String> producer = new KafkaProducer<>(props);  
producer.send(new ProducerRecord<>("healthcare-topic", "patient123", "heartRate:85"));  
producer.close();
```

*Spark SQL Example to analyze patient data:*

```
val spark = SparkSession.builder.appName("HealthcareAnalysis").getOrCreate()  
val df = spark.read.json("hdfs://path/to/healthcare_data.json")
```



```
// Average heart rate per patient  
df.groupby("patientId")  
  .agg(avg("heartRate").alias("avg_heart_rate"))  
  .show()
```

**Conclusion:**

The case study enhanced understanding of big data technologies in real-world contexts, linking theory with practice and highlighting challenges and benefits in industry applications.

**Learning Outcomes:**

- Students learn to relate big data theory to practical implementations.
- Students develop research and analytical skills.
- Students gain insight into the operational challenges and technological solutions in big data.

## **Experiment 10: Mini Project - Big Data Solution Design**

### **Aim:**

To design and implement a mini project using big data tools that demonstrate end-to-end data ingestion, processing, storage, and visualization.

### **Software Requirements:**

- Hadoop ecosystem (HDFS, MapReduce) or Apache Spark
- Hive for querying (optional)
- NoSQL database like MongoDB or Cassandra
- Visualization tools like PowerBI or Tableau

### **Theory:**

Big data projects require coordinating ingestion, storage, processing, and visualization technologies to handle data volume, variety, and velocity. This mini project helps consolidate learning by applying these technologies in an integrated workflow.

### **Procedure:**

1. **Define Problem Statement:**  
Select a real-world problem, e.g., analyzing social media sentiment or healthcare patient data trends.
2. **Design Architecture:**
  - Data ingestion (batch or streaming)
  - Storage (HDFS or NoSQL)
  - Processing (MapReduce or Spark)
  - Visualization (PowerBI/Tableau dashboard)
3. **Implement Components:**
  - Write MapReduce or Spark jobs for processing data
  - Store results in NoSQL or Hive
  - Prepare data extracts for visualization
4. **Create Dashboard:**  
Use PowerBI/Tableau to visualize processed data with charts and KPIs.
5. **Test & Document:**  
Run end-to-end pipeline and document project design, code snippets, results, and learning.

### Code/Syntax/Examples:

#### *MapReduce WordCount Mapper (Java):*

```
public class WordCountMapper extends Mapper<LongWritable, Text, Text, IntWritable> {  
    private final static IntWritable one = new IntWritable(1);  
    private Text word = new Text();  
  
    public void map(LongWritable key, Text value, Context context) throws IOException,  
        InterruptedException {  
        String line = value.toString();  
        StringTokenizer tokenizer = new StringTokenizer(line);  
        while (tokenizer.hasMoreTokens()) {  
            word.set(tokenizer.nextToken());  
            context.write(word, one);  
        }  
    }  
}
```

#### *Spark Python Example (PySpark):*

```
from pyspark.sql import SparkSession  
  
spark = SparkSession.builder.appName("MiniProject").getOrCreate()  
df = spark.read.csv("hdfs://path/to/dataset.csv", header=True, inferSchema=True)  
  
# Count patients by disease  
df.groupBy("disease").count().show()
```

#### *Hive Query Example:*

```
CREATE EXTERNAL TABLE healthcare (  
    patientId STRING,  
    age INT,  
    disease STRING,  
    admission_date STRING,  
    discharge_date STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE  
LOCATION '/user/hive/warehouse/healthcare/';  
  
SELECT disease, AVG(age) as avg_age  
FROM healthcare  
WHERE disease = 'Flu'  
GROUP BY disease;
```

#### *MongoDB Insert and Query:*

```
db.patients.insertOne({  
  patientId: "P001",  
  name: "John Doe",  
  age: 45,  
  diagnosis: "Diabetes",  
  admissionDate: ISODate("2024-05-01")  
})  
  
db.patients.find({ age: { $gt: 40 } })
```

### **Conclusion:**

The mini project demonstrated practical application of big data technologies in a cohesive workflow, reinforcing concepts of ingestion, processing, storage, and visualization.

### **Learning Outcomes:**

- Students gain hands-on experience with big data tools integration.
- Students learn to design and implement an end-to-end data pipeline.
- Students improve problem-solving and project documentation skills.